AI-Enhanced Daily Production Analytics Pipeline

Goal

To automate a daily review of the average cycle time per production line.

Approach

- Process and analyze daily data
- Detect performance changes
- Generate AI-assisted summary

Sources

Source Code:

https://github.com/zoe-zhousiyao/daily-production-analytics-siyao-2026.git

Demo Video:

http://demo-video-siyao.s3-website.eu-central-1.amazonaws.com

1 High-level architecture

Data Ingestion: csv-> S3 → **Processing: Lambda / Python** → **Orchestration: n8n** → **AI Layer: OpenAI** → **Presentation: S3 static Website**

2 Data flow

csv file

	timestamp	production_line	cycle_time_seconds
2	2026-03-08T08:01:12	Line_A	42
3	2026-03-08T08:02:10	Line_A	39
4	2026-03-08T08:03:01	Line_B	37
5	2026-03-08T08:04:21	Line_B	41
6	2026-03-08T08:05:11	Line_C	55
7	2026-03-08T08:06:45	Line_C	58

	timestamp	production_line	cycle_time_seconds
2	2026-03-09T08:01:12	Line_A	46
3	2026-03-09T08:02:10	Line_A	44
4	2026-03-09T08:03:01	Line_B	35
5	2026-03-09T08:04:21	Line_B	36
6	2026-03-09T08:05:11	Line_C	56
7	2026-03-09T08:06:45	Line_C	60

- The raw data is in CSV format.
- Each row represents a production cycle, and it is uploaded to S3.

Structured JSON


```
source_file:                "incoming/production_data_2026-03-08.csv"
processed_at:               "2026-04-20T07:04:06.711292"
invalid_rows:               0
▼ lines:
  ▼ 0:
      production_line:      "Line_A"
      average_cycle_time_seconds:  40.5
      completed_cycles:     2
      status:               "stable"
  ▼ 1:
      production_line:      "Line_B"
      average_cycle_time_seconds:  39.0  JS: 39
      completed_cycles:     2
      status:               "stable"
  ▼ 2:
      production_line:      "Line_C"
      average_cycle_time_seconds:  56.5
      completed_cycles:     2
      status:               "worsened"
```


- AWS Lambda is triggered.
- Python processed the CSV and calculates average cycle time per line.
- Result.json is generated.

Natural language prompt


```
OUTPUT   Schema   Table   JSON
1 item
[
  {
    "output": [
      {
        "id": "msg_0ca3b4fe84c05af70069f1afebe6f88193aad9fd836
              9048cdc",
        "type": "message",
        "status": "completed",
        "content": [
          {
            "type": "output_text",
            "annotations": [
            ],
            "logprobs": [
            ],
            "text": "- Overall: Production performance has
                    declined across some lines.\n- Best Line:
                    Line_B due to its improved average cycle
                    time of 35.5s.\n- Worst Line: Line_C due
                    to its increased average cycle time of
                    58s.\n- Action: Investigate and address
                    inefficiencies on Line_C."
          }
        ],
        "role": "assistant"
      }
    ]
  }
]
```

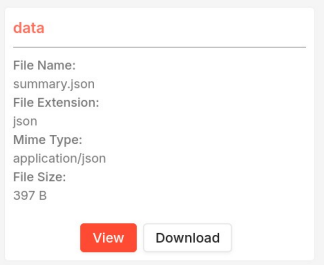

- n8n reads JSON from S3
- Extracts lines and status
- Generates prompt

AI-generated summary


```
{
  "id": "msg_041dde22336271ba0069f1b090236481a3b2116adb9f0d5ac2",
  "type": "message",
  "status": "completed",
  "content": [
    {
      "type": "output_text",
      "annotations": [
      ],
      "logprobs": [
      ],
      "text": "- Overall: Production performance has declined overall.\n- Best
              Line: Line_B due to the improved average cycle time of 35.5s.\n-
              Worst Line: Line_C because of the increased average cycle time of
              58s.\n- Action: Focus on reducing cycle time for Line_C through
              process optimization."
    }
  ],
  "role": "assistant"
```


- n8n sends prompt to OpenAI
- AI generates a daily production summary

Structured JSON



- n8n uses JavaScript to extract the AI output.
- n8n generates summary.json in n8n

Human-readable text

Daily Production Summary

- Overall: Production performance has declined overall.
- Best Line: Line_B due to the improved average cycle time of 35.5s.
- Worst Line: Line_C because of the increased average cycle time of 58s.
- Action: Focus on reducing cycle time for Line_C through process optimization.

- n8n uploads summary.json to S3
- used as dashboard data
- Dashboard displays the summary

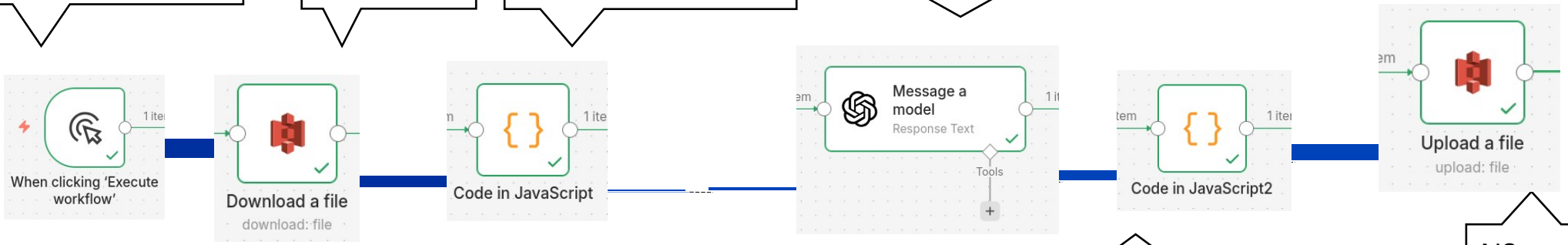
3 *Daily summary generation*

n8n orchestrates the pipeline and triggers AI summary generation.

n8n reads results.jso n from S3

The workflow transforms JSON data into a natural language prompt for OpenAI

n8n sends the prompt to OpenAI → AI generates a daily production summary → The OpenAI node uses a real API key, and each request consumes actual tokens.



The n8n workflow extracts the AI output and generates summary.json in n8n

N8n uploads summary.json to S3 → dashboard reads and displays it

Structured JSON: Result.json	Natural language prompt	AI-generated Summary: Summary.json	Structured JSON With AI-generated text	JSON with human-readable content

4 Detect worsening cycle time and additional consideration

Detect cycle time

1. Processed in AWS Lambda
2. Compare today_avg with yesterday_avg:
   if today_avg > yesterday_avg, it worsened
3. Compare with historical average:
   If today_avg > historical average, it worsened
4. Apply a threshold:
   if increase > 5%, it worsened

CSV file missing or arrives late

1. n8n checks S3 daily for the CSV file
2. If the file exists, continue processing;
   If not, wait and retry every 30 minutes,
   up to 3 times
3. If still missing, send an email to the team

Missing or invalid values in rows

1. Processed in AWS Lambda
2. Python validates each row
3. Invalid rows are skipped
4. The system logs the number of skipped rows

No data for a production line

1. Processed in AWS Lambda
2. If a line has no data for the day,
   do not calculate an average for that line
3. Mark it as “No data today”
4. Do not classify it as improved or worsened

5 *Keep the system simple and reliable*

Simplicity

- S3 and Lambda are both **serverless**, making the system easy to maintain.
- The system is **decoupled**, with each component responsible for a single task.

Reliability

- The System does not send raw CSV data directly to the AI.
- Instead, Python processes and structures the data first, reducing the risk of errors.
- It also handles invalid data and missing files, preventing the pipeline from failing or producing incorrect classifications.

Usability

- The system stores the final result in S3 and displays it through a **static dashboard**.
- The production team only needs to open a web page to see the daily AI summary.
- If n8n or OpenAI temporarily fails, the last successful **summary.json** remains in S3, so the dashboard can still show the latest available result.

Performance

- The system processes only one CSV file per day, keeping the design **lightweight.**
- This reduces OpenAI token usage and makes the workflow faster and more stable.